Ders 6 Genişletilmiş Notu: Command, Iterator, Memento ve Observer Kalıpları

Ders 6 Genişletilmiş Notu: Command, Iterator, Memento ve Observer Kalıpları

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Command Kalıbı
- 2. Command Örneği: Akıllı Ev Kumandası
- 3. Iterator Kalıbı
- 4. Iterator Örneği: Birleştirilmiş Yemek Menüsü
- 5. Memento Kalıbı
- 6. Memento Gerçeklemeleri ve Java Kodu
- 7. Memento Örneği: Geri Alma ve Yineleme
- 8. Observer Kalıbı
- 9. Observer Örneği: Müşteri İlişkileri Programı
- Örnek Sorular
- Dersin Toparlaması

1. Command Kalıbı

Geçen derste davranışsal kalıpların ilki olan Chain of Responsibility işlenmişti. Bu derse davranışsal kalıplardan Command ile devam edildi.

1.1 Amaç

Command kalıbının amacı, adından da anlaşıldığı gibi, çeşitli komutları yürütmektir. Ama hoca burada asıl meselenin şu olduğunu söyledi: nesnelere giden istekleri, yani metot çağrılarını (Dörtlü Çete'nin deyişiyle mesajları) birer nesne haline çevirmek istiyoruz. Başka bir deyişle metotları nesne yapmak istiyoruz. Bu sayede mesajlar kuyruğa alınabilir, güncesi (logu) tutulabilir; bu gibi işler yapılmaya başlanabilir.

1.2 GUI menüleri ve Dörtlü Çete'nin örneği

Hoca GUI programlayanların bu durumu bildiğini söyledi. Dosya menüsü altında menü seçenekleri ve onların alt seçenekleri vardır; birine, örneğin "Dosya Kaydet"e tıklanınca "gitsin şu nesnenin şu metodu çalışsın" diye kodlama yaparız. Kalıbın önerdiği çözüm, herhangi bir nesnenin herhangi bir metodu yerine belirli bir arayüzün belirli bir metodunu çalıştırmaktır. O arayüzün gerçeklemeleri sağlanarak işin nasıl yürütüleceği konusunda ince ayar yapılabilir.

Şemadaki durum şöyledir: bir uygulama vardır, uygulamanın menüsü vardır, menüye öğe (item) eklenebilir ve her öğenin bir tıklanma olayı vardır. Hocaya göre yalnız tıklanınca ne olacağını değiştirmek için çerçevenin (framework) `click` metodunun içini farklı biçimlerde güncellemeye izin vermesi yeter; bu, gereken en az davranıştır. Ama birden fazla komut söz konusuysa ve kuyruklama, günce tutma gibi ek işler isteniyorsa daha fazlası gerekir: bir `Command` arayüzü olur ve onun bir çalıştırma (`Execute`) metodu bulunur. Böylece bir menü öğesine

tıklanınca yürütülebilecek çok farklı komutlar, yalnız bu ortak arayüzü gerçekledikleri için yürütülebilir.

Örneğin bir menü seçeneği bir dosya açma komutuyla ilişkilendirilebilir. Soyut `Command` sınıfının somut alt sınıfı olarak bir `OpenCommand` yazılır; onun `Execute` metodunda yapılması gereken işler yapılır, gerekirse ek üye alanları ve metotlar eklenir. `OpenCommand`'dan uygulamaya (`Application`) geriye doğru bir bağımlılık vardır, çünkü dosya açma komutu çalışınca ana uygulamanın `Add` metodunu çalıştıracaktır. Hoca bunun kalıp için şart olmadığını, bir çeşitleme (varyasyon) olduğunu söyledi: komut, kendisini çağıran nesne ya da nesneler zinciriyle geri yönlü bir bağlaşıma sahip olabilir.

Başka bir olasılık olarak "Düzen > Yapıştır" (Edit, Paste) tıklanmış olabilir. O zaman `Execute` metodunun içinde o anda çalışılan belgeye `Paste` denir. Bu belge, az önceki nesne zincirinin (uygulama, menü, menü öğesi) bir parçası değildir.

1.3 Kalıp yapısı

Hoca iki örnekle şunu ayırt etmek istediğini söyledi: komutun yürütücüsü başka olabilir. Somut komutu (`ConcreteCommand`) bir işin yapılmasını başlatacak bir `Invoker` sınıfı mutlaka olacaktır. Ama invoker ile receiver aynı olmak zorunda değildir; hocaya göre bu kalıbın önemli bir noktasıdır. Hatta bir receiver olması da zorunlu değildir, receiver isteğe bağlıdır: somut komut sınıfı bir receiver'la uğraşabilir ya da işi kendi başına yapabilir. Hoca, üstteki `Command` bir arayüz olursa onun gerçeklemesine "alt sınıf" denemeyeceğini, bu yüzden "ilgili somut komut sınıfı" demeyi tercih ettiğini ekledi.

Kalıp yapısı (ekrandan, 00:17:10):

```
Client ------------------------------> Receiver
  :                                      Action()
  :                                        ^
  :      Invoker <>-----> Command          | receiver
  :                       Execute()        |
  :                          ^             |
  :                          |             |
  + - - - - - - - - - -> ConcreteCommand --+
                          Execute()   // receiver->Action();
                          state
```

- `Client`: receiver'ı tanır ve somut komutu oluşturur.
- `Invoker`: bir `Command` tutar ve onun `Execute()` metodunu çağırarak işi başlatır.
- `Command`: `Execute()` metodunu tanımlayan soyut sınıf ya da arayüz.
- `ConcreteCommand`: `Execute()`'u gerçekler, gerekirse `state` tutar; işi `receiver->Action()` ile receiver'a yaptırabilir.
- `Receiver`: işi asıl yapan nesne (isteğe bağlı).

Hoca olayı şöyle özetledi: `Execute` metodu üzerinden bir metodu nesne haline getirdik; `ConcreteCommand` sınıfından bir nesne verdik, "komut nesne oldu."

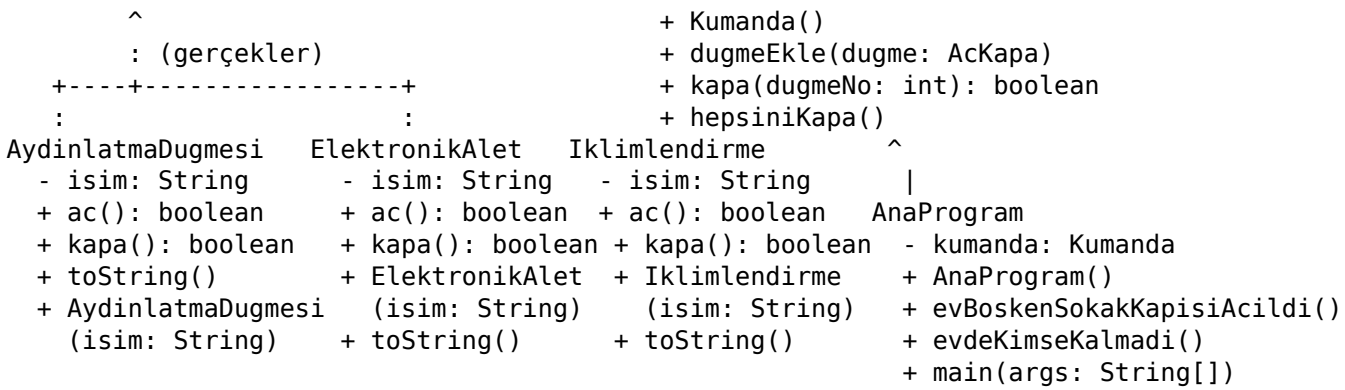
2. Command Örneği: Akıllı Ev Kumandası

2.1 Problem ve tasarım

Örnek, Head First Design Patterns kitabındaki bir örneğin küçük bir uyarlamasıdır (ekrandan, 00:20:00). Bir akıllı ev yazılımı üretiyoruz. Yazılımda basit bir açma/kapama kumandası ekranı vardır. Kumanda ekranındaki aç/kapa düğmelerine odaların ışıklarını, televizyonları, ısıtıcıları ve benzeri aygıtları açma/kapama işlevleri kazandırılacaktır.

Farklı bileşenlerin açılıp kapanması çok farklı biçimlerde olabilir, ama hepsinde açma ve kapama işlemi yapılacaktır. Bu yüzden açma ve kapama bir arayüzde tanımlanır. Slayttaki sınıf şeması:

```
<<interface>> AcKapa  <-----------*----  Kumanda
  + ac(): boolean                          - acKapaDugmeleri: ArrayList<AcKapa>
  + kapa(): boolean                        + ac(dugmeNo: int): boolean
```



Aydınlatma, elektronik alet ve iklimlendirme cihazı akıllı evin bileşenleridir ve kumanda nesnesi üzerinden kumanda edilebilirler; hepsi `AcKapa` arayüzünü gerçekler. Her birinin ne olduğunu anlatan bir adı vardır (“salonun elektrik düğmesi” gibi). Kumandada çok sayıda aç/kapa düğmesi olacaktır. Hoca bunları bir `ArrayList` içinde tutmayı tercih ettiğini, böylece düğmelere 0, 1, 2, 3 diye indisle erişilebildiğini (“üç numaralı düğmeyi kapat” gibi) söyledi.

`AnaProgram`’daki `evBoskenSokakKapisiAcildi()` ve `evdeKimseKalmadi()` gibi metotlar, ev boşken sokak kapısı açılınca ya da evde kimse kalmayınca ne yapılacağını tanımlar.

2.2 Kaynak kod

Hoca kaynak kodların zor olmadığını, yalnız derlenebilen ve birkaç temel mesaj veren bir iskelet kod hazırladığını söyledi:

- Her sınıfın kurucusu adını alır; her sınıfa sınıfın adını yazan bir `toString` eklenmiştir. Sınıflar şu an açık mı kapalı mı olduklarını tutmaz. Açıkken yeniden açılmaya çalışılınca `false` döndürmek gibi ayrıntılar eklenebilir, ama iskelette yoktur.
- `Kumanda`’nın kurucusu `ArrayList`’i ilklendirir. `dugmeEkle` basitçe listenin `add` metodunu çağırır.
- `ac(dugmeNo)` ilgili numaralı düğmeyi arar. Bulamazsa (null ise) hiçbir şey yapmadan `false` döndürür. Bulursa düğmenin `ac()` metodunun döndürdüğünü iletir; o da büyük olasılıkla `true` döndürür. `kapa` benzerdir. `hepsiniKapa` listedeki bütün elemanların `kapa` metodunu çağırır.
- `main` içinde kumanda paneline birkaç aydınlatma düğmesi, bir elektronik alet ve bir iklimlendirme cihazı (kombi) eklenir.
- Ev boşken sokak kapısı açılınca antrenin ışıkları yanar. Hoca örneği kışın yazmış olmalı ki kombinin de açılmasını eklemişti.
- “Evde kimse kalmadı” ana düğmesi kumandadaki bütün düğmeleri kapatır.

Hoca burada ekstra bir şey olmadığını, işin özünün komutları nesnelere çevirebilme yeteneği olduğunu, geri kalanın düz (straightforward) olduğunu söyledi.

> **Ek bilgi (derste anlatılmadı):** Örnekte `AcKapa` arayüzü `Command`, `ac()`/`kapa()` metotları `Execute()`, `Kumanda` ise `Invoker` rolündedir. Somut sınıflar işi ayrı bir receiver’a vermeden kendileri yaptığı için örnekte ayrı bir receiver yoktur; bu, hocanın “receiver isteğe bağlıdır” sözüyle uyumludur.

2.3 Aynı yapıya sahip kalıplar

Hoca bu örneğin sınıf şemasını çizerken önemli bir noktaya dikkat çekti. Bir arayüz, onu gerçekleyen sınıflar ve arayüzü kullanan bir sınıftan oluşan bu “standart” yapı birçok kalıpta görülür; ileride başka kalıplarda da görülecektir. Bu yapıya sahip kalıpları birbirinden ayıran şey, işlerin yürütülme ayrıntıları ve kalıpların odaklandığı amaçlardır.

> **Hocanın vurgusu:** Bu yapıya sahip kalıpların adları akılda kalsın. Bir problemde bunlardan biri en uygun olacak, diğerleri ise “yarı uygundur” diye anlatılabilir ve savunulabilir. Hoca bunu “önemli bir ipucu” olarak verdi.

Hoca kalıpla ilgili soru olup olmadığını sordu; soru gelmeyince Iterator’a geçti.

3. Iterator Kalıbı

3.1 Amaç

Iterator kalıbının amacı, parçalardan oluşmuş bir bileşenin parçalarını, bileşenin iç yapısını açığa çıkarmak zorunda kalmadan, istenilen (problemin gerektirdiği) herhangi bir biçimde dolaşmaktır (ekrandan, 00:30:00).

Farklı veri yapıları ve farklı veri yapılarında farklı dolaşma seçenekleri vardır. Bunların içinde dolaşıp bir şeyler yapmak isteriz, ama bunu yaparken parça-bütün yapısında hangi sınıflar ve hangi metotlar olduğunu tek tek aramak, sorgulamak zorunda kalmak istemeyiz. "Parçalardan oluşan bir bütün" denince akla Composite kalıbı gelebilir, ama illa Composite olması gerekmez; bir liste ya da ağaç gibi herhangi bir veri yapısı da olabilir.

Hoca farklı dolaşım biçimlerine örnekler verdi:

- İkili ağaçta sol-veri-sağ (LDR), sağ-veri-sol, veri-sol-sağ gibi üç harfin farklı sıraları. İkiden fazla çocuklu ağaçta da birinci sol ile birinci çocuğa, ikinci sol ile ikinci çocuğa gidilebilir, ama bu sıralar daha çok ikili ağaçlarda anlamlıdır.
- Tek yönlü bağlı listede tek bir dolaşma yönü vardır; çift yönlü bağlı listede geri de gidilebilir; dairesel bağlı listede sondan sonra ileri gidilince başa dönülür.

Bu farklılıklarla uğraşmak istemiyorsak ("bana hepsini getir, nasıl getirirse getirsin"), iterator'ı daha esnek ve rahat kodlayabiliriz. Iterator'ı kendimiz dolaştırırız; ileri git, geri git gibi işler önemli değilse ve yeter ki hepsi dolaşılsın deniyorsa, iterator'a sırayla "bana sonraki elemanı ver" diyebilmeliyiz.

Slayttaki örnek ve sorun (ekrandan, 00:30:00): örnek, bir veri yapısı içinde dolaşmaktır. Sorun, veri yapısının arayüzünü birçok farklı dolaşım türüyle şişirmek istememizdir.

3.2 İlk akla gelen yol ve sorunu

Akla ilk gelen yol, her veri yapısı için dolaşma metotlarını o sınıfın içinde yazmaktır. Farklı veri yapılarında farklı türden sınıflar bulunabilir; sınıf içindeki verinin türü değişti diye dolaşma metotlarını yeniden kodlamak gerekir ve ortaya birbirine benzeyen bir sürü kopya kod çıkar. Bir şeyi yanlış yaptığımızı fark etmeden bu dolaşma kodlarını başka veri yapılarına da ekleyebiliriz. Sonra bir gün hata fark edilince bütün veri yapılarında aynı düzeltmeyi yapmak gerekir. Hoca bunu "resmen bakım kâbusu" diye niteledi.

3.3 Java'nın Iterator'ı ve KISS ilkesi

Java'nın sınıf hiyerarşisinde bir `Collection` yapısı vardır. `Collection` uyumlu herhangi bir veri yapısından, elemanlarını gezmek için bir `Iterator` nesnesi istenebilir. Ama bu nesne Iterator tasarım kalıbına tam olarak uymaz; Java'nın Iterator'ı daha basittir (ekrandan, 00:30:00).

Hoca basit olmanın kötü olmak anlamına gelmediğini, birçok senaryoda Java'nın Iterator'ının işimize yaradığını ve kalıbı kendimizin kurmak zorunda olmadığımızı söyledi.

> **Hocanın vurgusu:** KISS ilkesi: "Keep it simple, stupid", yani "işi basit tut, aptal". Basitlik işimize yarıyorsa güzel bir şeydir; daha fazla karmaşıklaştırırsak aptallık etmiş oluruz. Java'nın Iterator'ı işinize yarıyorsa onu kullanın.

3.4 Birinci çözüm

Dörtlü Çete kalıbı anlatırken farklı çözümler üzerinden ilerler. Birinci çözümde bir `List` yapısı ve onun üzerinde dolaşacak bir `ListIterator` sınıfı vardır (ekrandan, 00:32:15):

```
List                                ListIterator
  Count()          <------list---     First()
  Append(Element)                     Next()
  Remove(Element)                     IsDone()
  ...                                 CurrentItem()
                                      index
```

Listenin eleman sayısını veren, araya ya da sona eleman ekleyen, eleman çıkaran metotları vardır. `ListIterator` `List` türünden bir üye ile listeye bağlıdır; listenin başına gitme, sonraki elemanı alma, daha fazla eleman olup olmadığına bakma, şu anki elemanı alma gibi metotları vardır.

- Çözümün güçlü yönü: veri yapısı ile dolaşım biçimi birbirinden soyutlanmıştır.
- Çözümün zayıf yönü: belli bir veri yapısı ile belli bir dolaşım biçimi arasında bağlılık oluşmuştur. Ağaç için `TreeIterator`, map için `MapIterator` gerekir. Bu bağlılığı azaltmak için genel bir liste yapısı arayüzü ve genel bir dolaşım biçimi arayüzü oluşturulabilir.

Hoca bir map'in örneğin `values` metoduyla bütün elemanların listesini döndürüp o liste üzerinde dolaşan bir iterator verebileceğini, bir ağacın ise özel dolaşım metotları için kendi `TreeIterator`'ını verebileceğini söyledi. Her veri yapısı hem herkesin kullanabileceği temel bir iterator (ileri git, sonraki eleman) hem de kendine özgü bir iterator döndürme yeteneğine sahip olabilir; kütüphane yazarken böyle şeyler yapılabilir.

3.5 İkinci çözüm

Farklı liste yapıları olacaktır: bağlı liste, çift bağlı liste, dairesel bağlı liste, iş parçacığı güvenli (thread-safe) ve güvenli olmayan listeler vb. Bunları ortak olarak kapsayacak bir soyut liste arayüzü ya da soyut sınıfı olsun. Bunda mutlaka bir `CreateIterator()` metodu olsun; bu metot bir iterator döndürsün. Iterator'da en temel olarak `First()`, `Next()`, `IsDone()` ve `CurrentItem()` olsun. Sonra soyut liste sınıfının somut alt sınıfları (`List`, `SkipList`) yazılır ve her biri kendi özel iterator'ını oluşturur (ekrandan, 00:40:00):

```
AbstractList <------ Client ------> Iterator
  CreateIterator()                    First()
  Count()                             Next()
  Append(Item)                        IsDone()
  Remove(Item)                        CurrentItem()
     ^      ^                           ^      ^
     |      |                           |      |
   List     |  <- - - - - - - - - ->  ListIterator
            |                                  |
         SkipList <- - - - - - - - -> SkipListIterator
```

Hocaya göre bu, önceki çözümün devamıdır: biraz daha esnemiş olsa da somut liste ile onun iterator'ı arasındaki bağımlılık yine vardır. Iterator kalıbı bizi buraya kadar getirir. Daha esnek ve ilginç işler için ileride görülecek Visitor kalıbı vardır; o, altyapısı daha karmaşık, daha "ağır siklet" bir kalıptır. Bu kötü bir şey değildir: KISS ilkesine göre, arkadaki somut liste gerçeklemesini bilmeden iterator üzerinden iş yapılabilir. Daha ayrıntılı işler gerekirse iterator'ı ilgili özelleşmiş alt sınıfın türüne çevirip onun özel işlemlerini kullanmak gerekir.

3.6 Kalıp yapısı ve bileşenleri

Kalıp yapısında veri yapısına `Aggregate`, onun somut türevlerine `ConcreteAggregate` denir (ekrandan, 00:36:50):

```
Aggregate <- - - - - Client - - - - - -> Iterator
  CreateIterator()                        First()
       ^                                  Next()
       |                                  IsDone()
       |                                  CurrentItem()
       |                                       ^
ConcreteAggregate  - - - - - - - - - - - >      |
  CreateIterator()  <------------------- ConcreteIterator
     // return new ConcreteIterator(this)
```

- `Aggregate` / `ConcreteAggregate`: parçaları arasında dolaşılacak nesnenin arayüzü / gerçeklemesi.
- (Gezgin) `Iterator` / `ConcreteIterator`: dolaşma işleminin arayüzü / gerçeklemesi.
- `java.util` paketindeki veri yapıları bu kalıbı kullanır.

Hoca `java.util` veri yapılarının kalıbı biraz daha basit ve biraz farklı biçimde kullandığını, farkın ne olduğunu ise şu an hatırlamadığını söyledi. Java'nın collection altyapısının başlı başına kalabalık bir konu olduğunu, yalnız bunu anlatan yeterince kalın Türkçe bir kitap gördüğünü de ekledi.

Hoca iterator için "gezgin" sözcüğünü ilk kez kullandığını, ama artık bu kalıbı "Iterator" diye anacağını söyledi.

3.7 Gerçekleme ayrıntıları

Kalıbı koda dökerken karar verilmesi gereken önemli noktalar vardır (ekrandan, 00:38:15). Hocaya göre bunlar konusunda karar verilmemişse o özellik desteklenmiyor demektir.

- Parçalar arasında dolaşırken değiştirme işlemine izin verilecek mi? Örneğin ilk eleman alınıp `next`, `next` diye ilerlenirken veri yapısındaki nesnelerin durum bilgileri değiştirilebilmeli mi? Normalde iterator ilgili nesnelerin kendisini gösterir, bu yüzden ek bir çaba harcanmazsa değişikliğe izin verilmiş olur. Iterator döndürülürken nesnelerin kendisi yerine klonları verilirse değişikliğe izin verilmemiş olur.
- Dolaşım algoritmasını kim gerçekleyecek ve dolaşılacak nesneler birer birer mi, yoksa hepsi birden mi istekçiye verilecek?
  - Gezgine komutlar verme sırasını (`first`, `next`, `next`, `isDone` …) istekçi (client) belirler ve parçaları gezginden tek tek okur.
  - İstekçi bir dolaşım istemini gezgine verir ve gezgin parçaları bir dizide döndürür; yani hepsi bir seferde istekçiye verilir.

"Dolaşım algoritmasını kim gerçekleyecek" sorusunun bir boyutu daha vardır: örneğin ikili ağaçta LDR ile mi, DLR ile mi dolaşılacak? Iterator oluşturulurken ilgili veri yapısına "bana LDR'ye göre dolaşacak bir iterator gönder" gibi bir komut verilebilir. En basitinden içinde çeşitli seçenekler olan bir enum kullanılabilir; hangisi verilmişse o sırada dolaşan altyapı ya bütünü oluşturan veri yapısında ya da iterator'da (hangisi işe daha çok yarıyorsa) kodlanabilir.

Hoca bu ayrıntıların birbirine göre bir avantajı ya da dezavantajı olmadığını, çözülecek problemin doğasıyla hangisi daha ilgiliyse onun kullanılacağını söyledi.

4. Iterator Örneği: Birleştirilmiş Yemek Menüsü

4.1 Problem

Örneğin kaynağı Head First Design Patterns'tir (ekrandan, 00:44:40). Hoca senaryoyu pandemide artan yemek siparişi ve eve teslimat işleriyle bağladı. Online yemek siparişi verilebilecek bir firma hızla kurulmak isteniyor. Hızlı girmek için hazır firmalar satın alınıyor: biri kahvaltılıkta, biri yemekte uzman iki firma.

- Menü elemanları (`MenuItem`) gerçeklemesi aynı denk gelmiştir, ama her firmanın kendi ayrı yemek menüsü gerçeklemesi vardır. Senaryoya inandırıcılık katmak için hoca iki firmanın aynı hazır (belki açık kaynak) çerçeveyi kullanmış olabileceğini, ama başka ayrıntılarda farklı kararlar verip farklı kodlamış olabileceklerini söyledi.
- Farklı menü gerçeklemelerinden yola çıkarak birleştirilmiş menü müşterilere ortak bir kod üzerinden nasıl sunulacak? İki firmanın yemekleri tek uygulamada, tek noktada gösterilmek isteniyor; başka firmalar da alınacak.
  - Ortak kod olmazsa her yeni satın alınan firma için menü sunma koduna yeni bir kod bloğu (yeni bir `if-else` ya da `switch-case`) eklemek gerekecek.
  - İşler daha da çetrefil hale gelebilir: iki firma da vejetaryenleri düşünüp hangi yemeklerin et içermediğini işaretlemiş. Bir vejetaryen menü sunmak için iki firmanın menüsünü ayrı ayrı dolaşıp et içermeyenleri göstermek gerekecek.
- Böylesi bir acemi çözümün kaynak kodu Eclipse'ten gösterildi. Hoca ders notlarını ve kaynak kodları web sayfasına koyduğunu, kötü çözümü slayta koyup notları şişirmek istemediğini söyledi.

4.2 Acemi çözüm (quick and dirty)

Eclipse'teki `dp.iterator.example` paketinde (ekrandan, 00:50:00) kahvaltılık firması `PancakeHouseMenu`, yemek firması `DinerMenu` adını taşır. `MenuItem` sınıfı ikisinde ortaktır; hoca normalde ayrı uygulamalarda menü öğelerinin de ayrı olacağını, ama işi karmaşıklaştırmamak için ortak tutulduğunu söyledi. İlk aşamada `implements Menu` kısmı da yoktur; onu sonra biz ekleyeceğiz.

- `DinerMenu` işleri bir dizi ile gerçeklemiştir. Dizide en fazla 6 eleman olabilir (`MAX_ITEMS`); yeni eleman eklenecekse sabit 7, 8 gibi değiştirilmelidir. Dizi dolunca `addItem` daha fazla ekleyemediğini söyler.
- `PancakeHouseMenu` işi "biraz daha iyi bilen" bir firma olarak `ArrayList` kullanmıştır; eleman ekleyip silmek daha kolaydır ve dizinin sabit uzunluk dezavantajından kurtulunur.
- Her iki menü de kurucusunda veri yapısını ilklendirip öğeleri ekler. Öğe; ad, açıklama, vejetaryen olup olmadığı ve fiyattan oluşur. Örneğin `DinerMenu`'deki iki öğe (ekrandan, 00:50:00):

```
addItem("Steamed Veggies and Brown Rice",
    "Steamed vegetables over brown rice", true, 3.99);
addItem("Pasta",
    "Spaghetti with Marinara Sauce, and a slice of sourdough bread",
    true, 3.89);
```

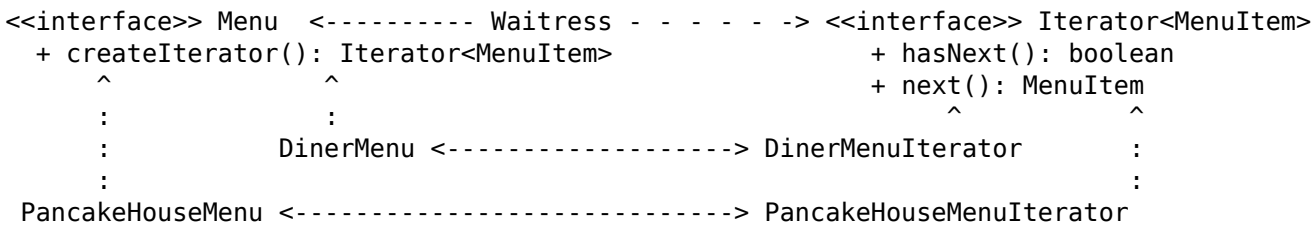
- `toString` yalnız menünün adını (örneğin `"Diner Menu"`) döndürür, bütün menüyü getirmez.
- Her iki menü de öğelerinin tümünü verebilir: `PancakeHouseMenu` listeyi, `DinerMenu` ise koskoca diziyi döndürür.

`WaitressQuickAndDirty` iki firmanın menülerini üye alan olarak tutar ve ilklendirir. `printMenu` bütün menüyü basar: önce "BREAKFAST" yazar ve `PancakeHouseMenu`'nün öğelerini (diğerinin kahvaltılığı yoktur) bir döngüyle tek tek yazar; sonra yemek seçeneklerini ("LUNCH") `DinerMenu`'nün dizisinden yazar. Hoca bunu yemek siparişi sitelerinin kahvaltılık, ara sıcak gibi farklı alt listeler sunmasına benzetti. Tek bir öğeyi yazan üç satır kopyala-yapıştır yapmak yerine `private` bir `printMenuItem` metodunda toplanmıştır; dışarıdan tek bir öğe yazdırmanın anlamı olmadığı ve iç çalışmayla ilgili olduğu için `private`'tır. Bu metot olmasaydı o satırlar her döngüde tekrarlanacak, en ufak değişiklik birden fazla yerde yapılacaktı.

Vejetaryen menü (`printVegetarianMenu`) yine önce kahvaltı sonra öğle yemeği biçimindedir; tek farkı öğeyi yazmadan önce öğenin vejetaryen olup olmadığına bakmasıdır. Yalnız kahvaltılıkları (`printBreakfastMenu`) ya da yalnız yemek menüsünü yazan metotlar da istenebilir. Hoca burada da kopya kod olduğunu, bu metotların `printMenu` içinden çağrılmasının daha iyi olacağını, ama bunun "quick and dirty" bir çözüm olduğunu ve ilk başta akla gelmediğini söyledi.

4.3 Iterator kalıbı ile çözüm

İşler iyi gitmeyince daha esnek bir altyapı için Iterator kalıbı kullanılır. Elemanlar zaten ortak olduğu için bu yapı rahat kurulur. Amaç, iki firmanın iki menüsünün de seçenekler arasında dolaşılabilecek bir iterator döndürebilmesidir. Firmalar artık bizim olduğu için kaynak kodlarına müdahale edebiliriz. Bir `Menu` arayüzü eklenir, mevcut sınıflar bu arayüzü gerçekleyecek biçimde değiştirilir ve bir iterator arayüzü tanımlanır: daha fazla eleman olduğu sürece sonraki elemanı getiren bir dolaşma kodlanır (ekrandan, 00:56:15):



`DinerMenu` `DinerMenuIterator` ile, `PancakeHouseMenu` `PancakeHouseMenuIterator` ile ilişkilidir. `Waitress` her ikisinin iterator'larını alıp ortak arayüz üzerinden "tıkır tıkır" listeleyecektir; hoca bunu garsonun birinin mutfağına gidip ayrı, öbürünün mutfağına gidip ayrı içerik almak zorunda kalmamasına benzetti.

`Waitress` yine iki menüyle ilişkilidir ve bunları kurucuda alır. Hoca kurucuya doğrudan iterator'lar da verilebileceğini, üye alan olarak menüler yerine iterator'lar tutulabileceğini söyledi. Ama o zaman `Waitress`'ı çalıştırmak için ilgili iterator'ları alıp ona verecek bir ön yapılandırma sınıfı eklemek gerekirdi. Dışarıya verilen hizmeti ve arayüzü değiştirmemek, geriye doğru uyumluluğu korumak için bu yapılmadı. `Waitress` menüleri bütün olarak alsa da onların üzerinde doğrudan dolaşmaz.

`printMenu` önce kahvaltı menüsünün iterator'ını ister, ekrana "BREAKFAST" yazar ve işi iterator alan `printMenu(Iterator)` metoduna verir; sonra yemek menüsünün iterator'ını alıp aynı metodu çağırır. Bu metot yalnız şunu söyler: üzerinde çalışılacak eleman olduğu sürece o elemanı getir ve menüye yaz. Menü öğesini yazan yardımcı metot öğenin adını, fiyatını ve açıklamasını yazar (ekrandan, 01:00:00):

```
private void printMenuItem( MenuItem menuItem ) {
    System.out.print(menuItem.getName() + ", ");
    System.out.print(menuItem.getPrice() + " -- ");
    System.out.println(menuItem.getDescription());
}
public void printVegetarianMenu() {
    System.out.println("\nVEGETARIAN MENU\n----\nBREAKFAST");
    ...
}
private void printVegetarianMenu(Iterator<MenuItem> iterator) {
    while (iterator.hasNext()) {
        ...
    }
}
```

Vejetaryen menü için yalnız vejetaryenlere özgü bir iterator kodlamak yerine bu karar yine `Waitress`'a bırakılmıştır: kahvaltı ve yemek menülerinin iterator'larını alır, vejetaryen menü yazan metot öğeyi yazmadan önce vejetaryen olup olmadığına bakar.

Hoca `printMenu(Iterator)` ile `printVegetarianMenu(Iterator)` metotlarının birbirine benzemesini kopyala-yapıştır kod olarak görmediğini, bunun iterator'ın olması gereken basit dolaşma kodu olduğunu söyledi. Dolaşma mantığı değişmez ("dönülecek eleman var mı? Varsa yaz, sonra sıradakini getir"); bir işin ayrıntısı değişirse yalnız ilgili metotta değişiklik yapılır. Kahvaltı ve yemek menülerini ayrı yazan metotlar için ise önceki eleştiri yine geçerlidir; hoca bunu bir sonraki refactoring yinelemesine bıraktı.

4.4 İki çözümün karşılaştırılması

Hoca Eclipse'in karşılaştırma aracıyla `Waitress` ile `WaitressQuickAndDirty`'yi yan yana koydu ve kazançları şöyle sıraladı:

- Eski halinde iki ayrı menüyle tek tek uğraşmak gerekirken artık ikisi ortak arayüz altında toplanmıştır.
- Eski halinde menülerden birinin altyapısında `ArrayList`, öbüründe dizi olduğunu bilip ona göre kod yazmak gerekirken artık ikisine de eş biçimli erişilir.
- Listede ve dizide dolaşmayı ayrı ayrı kodlamak yerine eş biçimli dolaşma tek bir metotta toplanmıştır. Dolaşma için gereken komut sayısı azalmış, daha az karmaşık bir yapıyla daha temiz kod elde edilmiştir; bakım masrafı daha az olacaktır.

4.5 Çalıştırma

Aradan sonra hoca bir ana program ekleyip çalıştırdı (ekrandan, 01:20:00):

```
package dp.iterator.example;

public class MainProgram {
    public static void main(String[] args) {
        Waitress w = new Waitress(new PancakeHouseMenu(), new DinerMenu() );
        //WaitressQuickAndDirty w = new WaitressQuickAndDirty( );
        w.printMenu();
```

```
    }
}
```

Çıktı (ekrandan, 01:40:00 ve 01:20:00):

```
MENU
----
BREAKFAST
K&B's Pancake Breakfast, 2.99 -- Pancakes with scrambled eggs, and toast
Regular Pancake Breakfast, 2.99 -- Pancakes with fried eggs, sausage
Blueberry Pancakes, 3.49 -- Pancakes made with fresh blueberries
Waffles, 3.59 -- Waffles, with your choice of blueberries or strawberries

LUNCH
Vegetarian BLT, 2.99 -- (Fakin') Bacon with lettuce & tomato on whole wheat
Regular BLT, 2.99 -- Bacon with lettuce & tomato on whole wheat
Soup of the day, 3.29 -- Soup of the day, with a side of potato salad
Hotdog, 3.05 -- A hot dog, with saurkraut, relish, onions, topped with cheese
Steamed Veggies and Brown Rice, 3.99 -- Steamed vegetables over brown rice
Pasta, 3.89 -- Spaghetti with Marinara Sauce, and a slice of sourdough bread
```

Kahvaltıda 4, öğle yemeğinde 6 öğe yazdırılır. Vejetaryen menü de çalıştırıldı ve yalnız vejetaryen öğeleri yazdırdı. Hoca vejetaryen ile veganı ayırdı: vegan hayvansal hiçbir şey yemez, örneğin yumurta yemez; ama vejetaryen yumurta yiyebilir. Bu yüzden yumurtalı ve tostlu kahvaltı da listede çıktı. Hoca listede pancake kahvaltısını, Blueberry Pancakes'i, Waffles'ı, adında "Vegetarian" geçen öğeyi ve makarnayı (pasta) saydı.

Eski ve yeni çözüm arasında kurucu farkı da vardır: eski `Waitress` hiçbir şey almaz, firmaların menülerini kendisi oluşturur; yeni `Waitress`'a menüler verilir. Hocaya göre hangisinin daha iyi olduğu fark etmez, ama yenisi daha esnektir: üçüncü, dördüncü, beşinci yemek firmaları da alınıp farklı alt markalar altında (birinci ile ikinci, birinci ile üçüncü gibi) kümelenecekse, her küme için ayrı kod yazmak yerine "al şu menüleri, bunlarla uğraş" diyen tek bir kod yazılır. Ama bu haliyle her yeni firmada kurucuya parametre ekleyip imzayı değiştirmek gerekir. Bunun yerine kurucu, `Menu` arayüzünü gerçekleyen menülerden oluşan bir liste ya da Java'nın değişken sayılı parametre (`...`) sözdizimiyle istenen sayıda menü alabilir. Hoca bunu C'deki `printf`'e benzetti: ilk parametredeki biçim belirteçlerine (`%s`, `%d`) göre virgülle istenen sayıda parametre verilir.

> **Ders sonrası not:** Hoca değişken sayılı parametrelerin Java 1.6 ile geldiğini tahmin etti. Bu özellik büyük olasılıkla Java 5 (1.5) ile gelmiştir.

5. Memento Kalıbı

5.1 Adın anlamı

Hoca aynı adlı ilginç bir film olduğunu anlattı: başrol oyuncusunun adını hatırlamadı, ikinci rolde Carrie-Anne Moss vardı. Filmde bir travmadan sonra yakın dönem hafızasını kaybeden ve kız arkadaşını öldüren kişiden intikam almak isteyen bir adam anlatılır; film olayları şimdiden geçmişe doğru anlatır. "Memento" hatıra, anı demektir: gittiğimiz yerlerden aldığımız hediyelikler, çektiğimiz resimler, arkadaşımızın hediye ettiği şeyler gibi.

> **Ek bilgi (derste anlatılmadı):** Film büyük olasılıkla Christopher Nolan'ın 2000 yapımı "Memento" filmidir; başrol oyuncusu büyük olasılıkla Guy Pearce'tır.

5.2 Amaç ve kullanım alanları

Memento kalıbının amacı, bir nesnenin durum bilgisini sarmalama (encapsulation) ilkesini bozmadan elde etmektir. Bir nesnenin durum bilgisi, üye alanlarının tümünün belirli bir andaki değeridir. Bunu nesnenin kendisinden elde ederiz, ama belirli amaçlarla almak isteriz; hatta durum bilgisinin yalnız bir alt kümesiyle ilgileniyor olabiliriz. Kalıp bu esnekliği de sağlar.

Durum bilgisinin parçalarına neden ihtiyaç duyulabilir:

- Nesneyi ikincil saklama ortamlarına yazmak.

- Ağ üzerinden başka bir bilgisayara aktarmak, orada uzak bir kopyasını oluşturmak.
- Yapılan işlemleri geri alma (undo) düzeni eklemek: nesne belli bir duruma getirilir, o anki durumu saklanır, bazı işlemler yapılır.
- Hareket (transaction) mantığı: bir hareket birden fazla alt adımdan oluşur ve bu adımların hepsinin birlikte başarılı olması gerekir. Arada bir sorun olursa yapılan değişiklikler atılır ve nesne işlerin başlamasından önceki güvenli haline döner.
- Üzerinde çalışılan bir belgede geri alma ve yineleme (undo/redo) işlemleri.

Durum bilgisinin yalnız bir kısmının saklanması gerekiyorsa kalıp daha da avantajlıdır: bütün durumu içeren nesnenin çok sayıda kopyası yerine yalnız ilgilenilen alt kümeyi içeren nesneler biriktirilir ve bellek ya da disk tasarrufu yapılır.

Sorun şudur: sarmalamayı bozmak istemiyoruz. Bir nesnenin üye alanları ve metotlarının bütünlüğünü ve veri gizlemeyi korumak istiyoruz; üye alanlarını `public` yapamayız, `private` kalmalarını istiyoruz. Kalıp bunun bir yolunu sunar: istenilen anlarda nesnenin mementosu alınır, mementolar biriktirilir; duruma göre geri alma yapılır ya da memento başka bir bilgisayara gönderilir.

5.3 Kalıp yapısı

Kalıp yapısı (ekrandan, 01:30:00):

```
Originator                        Memento   <---memento---<> Caretaker
  SetMemento(Memento m) - - - ->    GetState()
  CreateMemento()                   SetState()
  state                             state

  CreateMemento():  return new Memento(state)
  SetMemento(m):    state = m->GetState()
```

- `Originator`: durum bilgisiyle ilgilenilen, durum bilgisinin tamamı ya da bir alt kümesi saklanacak nesne.
- `Memento`: saklanacak durum bilgisini tutar. Slayttaki `state` tek bir üye gibi gösterilmiştir, ama birden fazla üye olabilir. Originator'daki durumun yalnız bir alt kümesiyle ilgileniyorsak Memento'ya yalnız o üye alanlar yazılır. `state` için getter ve setter metotları vardır.
- `Caretaker`: gerekli anlarda durum bilgisinin bir kopyasını alır ve elindeki kopyalardan birini geçerli durum haline getirir.

Originator'ın `CreateMemento` metodu çağrılınca kendi durum bilgisini saklayan bir Memento nesnesi elde edilir. `SetMemento` metoduna bir Memento verilince nesne o duruma geçer (geri döner ya da yinelenir). Dörtlü Çete metotların içinin nasıl kodlanacağını notlarla göstermiştir: `CreateMemento`, `state`'i kurucuya parametre olarak vererek yeni bir Memento döndürür; `SetMemento` ise Memento'nun `state`'ini alıp nesnenin `state`'ine yazar.

Caretaker mementoların bir listesini tutabilir; şemadaki elmas bunu gösterir. Bu bir liste, map, ağaç ya da yığın olabilir. Hoca yığın için derleyicileri örnek verdi: fonksiyon çağrıları yapıldıkça mevcut kapsam bilgisi yığına atılır, her `return`'de yığından çekilir.

Hoca bunun kolay kodlanacağını, tasarım kalıplarının zaten zor kodlanacak bir şey vermeye çalışmadığını söyledi. Asıl sorun şudur: sarmalamayı bozmak istemiyoruz ve dışarıdan herhangi bir üçüncü tarafın, hatta Caretaker'ın kendisinin, sahte bir memento oluşturup Originator'a "senin durumun bu" diye verebilmesini istemiyoruz.

5.4 Etkileşim şeması

Caretaker, Originator'a "bana memento ver" (`createMemento`) der. Originator yeni bir Memento oluşturur, durumunu ona yazar (`setState`) ve Memento'yu Caretaker'a geri verir. Daha sonra orijinal hale dönülmek istenince Caretaker daha önce almış olduğu mementoyu `setMemento` ile Originator'a verir; Originator mementonun durumunu (`getState`) alır, kendi ilgili setter metotlarıyla yazar ve ancak o zaman denetim akışını geri döndürür.

Hoca Dörtlü Çete'nin etkileşim şemasında geri dönüş oklarının hiçbirinin gösterilmediğini, kendisinin bunları eklediğini söyledi. Ayrıca şemadaki yeni nesne oluşturma (`new`) çağrısının

kesik çizgi değil, yeni standarda göre düz çizgi olması gerektiğini, dönüş değeri yazmaya gerek olmadığını, parametreleri üç nokta ile göstermeyi tercih ettiğini belirtti.

> **Ders sonrası not:** UML 2 belirtiminde oluşturma (create) mesajı büyük olasılıkla kesik çizgi ve açık uçlu okla çizilir. Hocanın "düz çizgi olmalı" sözü şemadaki başka bir oka yönelik olabilir; slayt görülmeden kesinleştirilemedi.

5.5 Ne zaman kullanılır

Kalıp, bir nesnenin durum bilgisinin tümünü ya da bir kısmını daha sonra o duruma dönebilmek için saklamak gereken her durumda kullanılabilir. Ama kalıbı kurmanın çabaya değmesi için bunu veri gizliliği ilkesini bozmadan yapmak istemek gerekir. İstek şudur: Caretaker mementoyu tutabilsin ama değiştiremesin. Memento'nun `GetState` ve `SetState` metotlarına Caretaker ya da başka bir sınıftan bir nesne ulaşamamalı, yalnız Originator ulaşabilmelidir. Aksi takdirde dışarıdan herhangi biri ya da Caretaker sahte bir durum verebilir.

Nesneye yönelimde genel olarak `public` ve `private` kuralları vardır: `public` tanımlanan metodu herkes çağırabilir, `private` tanımlananı başka kimse çağıramaz. "Bir sınıfın belli bir metodu yalnız şu şu sınıflardan çağrılsın" diye bir liste verilemez. Peki ne yapılabilir?

6. Memento Gerçeklemeleri ve Java Kodu

6.1 C++: friend sınıf

C++'ta `Memento` sınıfının başına `friend Originator` yazılabilir. Bir sınıf diğerinin friend'i olarak tanımlanırsa, friend olan sınıf diğer sınıfın `private` metotlarına erişebilir. Böylece Memento'nun durum metotlarını yalnız Originator çağırabilir.

6.2 Java: paket erişimi

Java'da böyle bir özellik yoktur. Paket (package) erişim kuralı kullanılabilir: Memento'nun metotlarına `public` ya da `private` denmez. O zaman bu metotlar paket erişimine sahip olur: aynı paketteki diğer sınıflar erişebilir, farklı paketlerdeki sınıflar erişemez. Originator ile Memento tek bir pakette tutulur ve o pakete başka hiçbir sınıf eklenmez.

Hoca bunun daha sakıncalı olduğunu söyledi: kod dağıtılırken, yani jar dosyası oluşturulurken paketlerin mühürlenmesi (seal) gerekir. Aksi takdirde herhangi biri o pakete kendi sınıfını ekleyip bütün paket metotlarını istediği gibi kullanabilir. Bu yüzden paket erişimi pek işe yaramaz, ya da en azından dikkatli olmak gerekir.

Derste paket erişimli çözüm (`dp.memento` ve `dp.mementoOuter` paketleri) gösterildi. Originator kurucuda durumu ilklendirir; `createMemento` bir Memento oluşturur, onun durumunu kendi durumu yapar ve Memento'yu döndürür. Memento'nun `private` bir `state` üyesi, onun için getter ve setter metotları vardır. `State` sınıfı `durum` adlı bir `String` içerir ve getter/setter metotları vardır. Memento'nun getter metodu `private` yapılamaz, paket erişimli bırakılmalıdır. Hoca setter'ı da `private` yapmayı denedi: ayrı paketteki Caretaker'da derleme hatası alındı, bu yüzden metotlar paket erişimli kalmak zorundadır. Bu da, isterse pakete eklenebilecek herhangi bir sınıfın durumu alıp değiştirebileceği ve sahte durumu `setState` üzerinden mementoya yazabileceği anlamına gelir; bütün amaç zaten bunu önlemektir.

Hoca bu sırada Eclipse'te kod kalitesi eklentilerinin verdiği uyarılar ve bir derleme sorunuyla bir süre uğraştı; sorun Memento ile ilgili değildi.

6.3 Java: iç sınıf çözümü

Java'da (ve iç sınıf mantığını destekleyen bütün dillerde) başka bir seçenek vardır: Memento, Originator'ın iç sınıfı (inner class) yapılır. Ama bu durumda unutulmaması gereken bir şey vardır: Originator'ın `createMemento` metodunda durum klonlanıp mementoya öyle verilmelidir. Hoca ilk önce iç sınıf çözümünü kodladığını, ama testlerinde istediği sonucu bir türlü alamadığını, araştırınca çözümün durumun klonlanması olduğunu bulduğunu ve bunu paylaşmak istediğini söyledi.

İç sınıf çözümünde (`dp.mementoV2.innerClass`) Memento ve diğerleri aynı pakette olabilir; ayrı bir pakete gerek yoktur. Originator'ın içinde, Originator kapanmadan önce `public class Memento` tanımlanmıştır. Originator'da `State` türünden bir durum üyesi, onu ilklendiren kurucu, durumu değiştiren bir `modify` metodu, `toString` ve `setMemento` vardır. `createMemento`, öncekinden farklı olarak Memento'nun durumunu atarken durumu klonlar. İç sınıfın kodu (ekrandan, 01:50:00):

```
public String toString() {
    return state.getDurum();
}

public class Memento {
    private State state;

    private State getState() {
        return state;
    }

    private void setState(State state) {
        this.state = state;
    }
}
```

Tek fark şudur: Memento iç sınıf olduğu için `getState` ve `setState` rahatlıkla `private` yapılabilir; böylece Originator dışında kimse onlara erişemez. `State` sınıfı aynıdır, ama klonlama gerektiği için `Cloneable` arayüzünü gerçekler ve bir `clone` metodu vardır. `clone` kendisini döndürmez; aynı `durum` dizgisine sahip yeni bir `State` nesnesi oluşturup onu döndürür.

Caretaker başka bir pakettedir (`dp.mementoV2Outer`). Mementoları biriktirebilmek için Originator'ın Memento iç sınıfını import etmesi gerekir; bu, Java'nın iç sınıf yapısından kaynaklanır. Test kodu (ekrandan, 02:00:00):

```
package dp.mementoV2Outer;
import dp.mementoV2.innerClass.*;
import dp.mementoV2.innerClass.Originator.Memento;

public class Caretaker {
    public void test( ) {
        Originator subject = new Originator( );
        subject.modify("ilk durum");
        Memento mem = subject.createMemento( );
        subject.modify("yeni durum");
        //bir süre sonra...
        subject.setMemento( mem );
        System.out.println(subject);
    }
    public static void main( String[] args ) {
        Caretaker ct = new Caretaker();
        ct.test();
    }
}
```

Önce durum "ilk durum" yapılır, memento alınır, durum "yeni durum" olarak değiştirilir; bir süre sonra geri alma yapılır, yani eski memento verilir. Ekrana "yeni durum" değil "ilk durum" yazılması beklenir. Çalıştırılınca çıktı `ilk durum` oldu (ekrandan, 01:50:00).

Hoca klonlama yapılmazsa ekrana hatalı olarak "ilk durum" yerine "yeni durum" çıktığını, klonlamanın bunu düzelttiğini söyledi.

> **Ek bilgi (derste anlatılmadı):** Klonlama yapılmazsa memento, Originator'ın kullandığı `State` nesnesinin kendisine bir başvuru tutar. `modify` bu ortak nesnenin içindeki dizgiyi değiştirdiği için mementodaki durum da değişmiş olur ve geri dönüş işe yaramaz. Klon, o anki durumun ayrı bir kopyasını sakladığı için sonraki değişiklik-

lerden etkilenmez.

Elde edilen mementonun üzerinde hiçbir değiştirme metodu çağrılamaz; dışarıdan yalnız `Object` sınıfının metotları görülür. Hocaya göre teoride memento güvenlidir: dışarıdan, aynı paketteki bir sınıftan bile müdahale edilemez. Yani mevcut bir mementoyu alıp sahte hale getirme sorunu çözülmüştür.

6.4 Sahte memento sorunu

Hoca bu çözümün yeterince güçlü olup olmadığını sınamak için onu "kırmayı" denediğini anlattı. Slayttaki açıklamada yazan deneme şudur (ekrandan, 02:00:00):

```
Originator fakeSubject = new Originator( );
fakeSubject.modify("sahte durum");
Memento memFake = fakeSubject.createMemento();

subject.setMemento( memFake );
```

Aynı sınıftan sahte bir Originator oluşturulur, durumu "sahte durum" yapılır, ondan bir memento alınır ve bu memento asıl nesneye verilir. Bu kabul edilir. Hoca, bu kırma denemesini aslında Memento üzerinden yapamadığını, sahte Originator'dan alınan mementonun kalıp açısından geçerli bir memento olduğunu söyledi. Bunun engellenmesi için bir Originator'a yalnız kendisinden kaynaklanan bir memento verilebilmelidir.

Hoca iki fikir söyledi:

- Mementoların saklanmasını (idamesini) Caretaker'a değil Originator'ın kendisine vermek; Caretaker `setMemento` yerine `activateMemento(int siraNo)` diyerek nesneyi o sıradaki duruma döndürür. Hoca bunun pek tercih edilecek bir durum olmadığını, çünkü bütün bu idareyi Caretaker'ın yapmasını istediğimizi, problemin özünün bu olduğunu söyledi. Bu dönemde bunu tamamlayamadığını, çözümün mükemmel olmadığını açıkça belirtti.
- Nesnenin kimliğini mementonun içine gömmek: Originator bir mementoyu yalnız kendi kimliğinden oluşturulmuşsa kabul eder, uyumsuz kodlu mementoyu reddeder. Ama Java sanal makinesinin nesnelere verdiği evrensel benzersiz kimliklere programcının erişme yolu yoktur; ancak `hashCode` üzerinden yapılabilir. Bu çözüm yalnız program çalıştığı sürece geçerlidir: program kapatılmadan önce mementolar diske saklanırsa, program yeniden açıldığında yeni Originator'ların `hashCode` değerlerinin ne olacağı belli olmadığı için o mementolar geçersiz hale gelir. Yani kalıcı bir çözüm değildir.

Hoca konuyu burada bırakıp daha fazla didiklemek istemediğini söyledi.

7. Memento Örneği: Geri Alma ve Yineleme

Çözülecek problem: bir belge işleme programında geri alma ve yineleme işlemleri (ekrandan, 02:02:20).

```
Controller 1 ------*--> Document                          DocumentMemento
   :                    - currentState: DocumentMemento    - documentState
   : (DocumentMemento   - states: List<DocumentMemento>        ^
   :  ile hiçbir şekilde - position: int                        | *
   :  ilişkisi yok)     + undo()        -------1----------------+
                        + redo()
                        + anyOperation()
```

```
public void undo( ) {
    position -= 1;
    currentState = states.get(position);
}
public void redo( ) {
    position += 1;
    currentState = states.get(position);
}
```

```
public void anyOperation( ) {
    states.add(currentState);
}
```

Belgenin bir belge mementosu olur. `Controller` herhangi bir anda belgeye "geri al" ya da "yinele" diyebilir; dikkat edilirse `Controller`'ın `DocumentMemento` ile hiçbir ilişkisi yoktur. Geri alma tek adımlık ya da çok adımlık olabilir. Hocaya göre çok adımlık olacaksa ya biraz daha ek kod yazılır ya da 10 adım geri dönmek için 10 kez `undo` yapılır. Bunun için bir `position` durumu tutulur.

`position` sıfırdan daha aşağı inmemelidir; slaytta böyle bir denetim kodlanmamıştır. Hoca derste düzeltmeyi şöyle söyledi: `position` sıfırdan büyükse `position--` yapılır.

Slayttaki açıklamaya göre bu örnek "biraz yarım" bir Memento uygulamasıdır, çünkü Caretaker rolünü `Document` üstlenmiştir. Hoca basit bir uygulama yapmak istediğini, bunun dert olmadığını söyledi.

8. Observer Kalıbı

Hoca kalan kalıpları gözden geçirdi ve Observer kalıbını da işleyip dersi bitirmek istediğini, kalıbın çok basit olmadığını ama süreyi aşmayacağını söyledi.

8.1 Amaç ve örnek

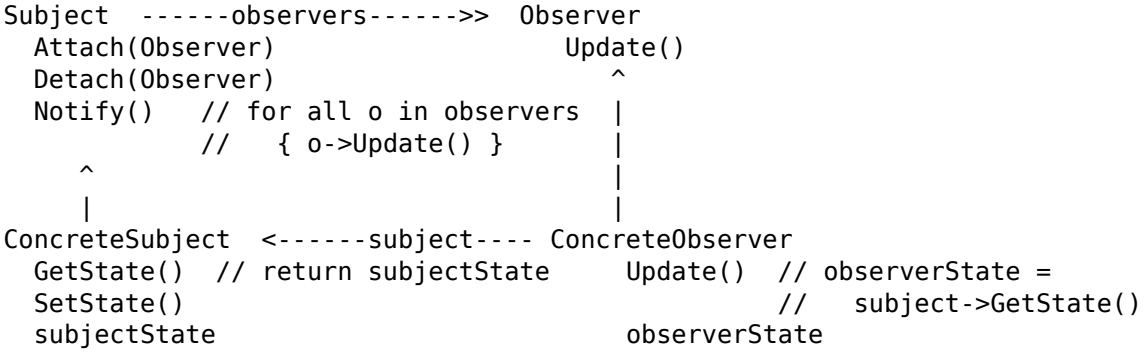
Observer kalıbının amacı, bir nesnenin durumu değiştiğinde onunla ilişkide bulunan tüm diğer nesnelerin ya da yalnız bir alt kümesinin haberdar edilmesini ve güncellenmesini sağlamaktır. Bu ilişkiler sabit kodlanmış (hard-coded) değildir; "ekle, çıkar, şu şunu izlesin, şu izlemesin" gibi ekleme ve çıkarmalarla çalışma anında kurulur. Bu yüzden asıl problem, böyle bir durumda haberdar etme işinin nasıl yürütüleceğidir.

Örnekte bir hesap tablosunda bir veri vardır ve bu verinin grafikleri çizilmiştir. Veri değişince grafiklerin de kendiliğinden güncellenmesi istenir.

Verisiyle ilgilenilen nesne `Subject`'tir ve onun alt sınıfları olabilir. Tek bir subject olsa sorun olmaz, ama durum bilgisiyle ilgilenilen farklı türden nesneler olabilir ve ilgilenen ile ilgilenilen arasında farklı eşleşme türleri bulunabilir. Kalıp bunu da destekleyen bir çözüm önerir.

8.2 Kalıp yapısı

Kalıp yapısı (ekrandan, 02:07:50):



- `Subject`: durumundaki değişikliklerin izleneceği nesnenin arayüzü. Birden fazla gözleyicisi olabileceği için gözleyici ekle/sil metotlarını tanımlayan soyut bir sınıf olmalıdır.
- `Observer`: gözleyici nesnelerin arayüzü.
- `ConcreteSubject` ve `ConcreteObserver`: bunların somut alt sınıfları.

Hoca `Subject` için arayüz değil soyut sınıf kodlamak gerektiğini vurguladı: subject'in gözleyicilerini haberdar edebilmesi için gereken işlevler bu soyut sınıfa kodlanır ki alt sınıflarda tekrar tekrar kodlanmasın.

Çalışma biçimi şöyledir: `Subject`'e gözleyici ekleyen (`Attach`) ve silen (`Detach`) metotlar vardır. Bir de `Notify` metodu vardır; subject'in durumu değişince gözleyicilerin haberdar edilmesini

sağlar. `Notify`, gözleyici listesindeki her gözleyicinin `Update` metodunu tek tek çağırır; "bu kadar basit." `Update` çağrılan gözleyici, izlediği varlığa gidip "senin durumun neymiş, bir alayım" der; yani durum değişikliğinden haberdar olunca yeni durumu subject'e sorar.

`ConcreteObserver`'dan `ConcreteSubject`'e giden `subject` başvurusu her zaman kurulmak zorunda değildir; isteğe bağlıdır. Durum bilgisi gözleyicilere gönderilirken bütün durum bilgisi gönderiliyorsa, gözleyicinin subject'e "neyin değişti, durumunu bir alayım" demesine gerek yoktur; bilgi zaten üstteki soyut sınıflar arasındaki ilişki sayesinde somut gözleyiciye gitmiş olabilir.

8.3 Push ve pull modeli

Hoca bütün bilginin gözleyiciye gönderildiği duruma push modeli, alternatifine pull modeli dendiğini söyledi. Slayttaki gerçekleme ayrıntıları (ekrandan, 02:10:00):

- Pull modeli:
  - (+) Gözlenen yalnız gözleyicileri haberdar eder, gözleyiciler ilgili durum bilgisini gözlenenden alır.
  - (+) Gözlenenin gözleyicilerle bağlantısını azaltan çalışma biçimidir.
  - (+) Gözleyici durum bilgisinin yalnız istediği kısmını almayı seçebilir.
  - (−) Gözleyici gözlenende nelerin değiştiğini kendisi çıkarmak zorunda kalır.
- Push modeli: gözlenen gözleyicilere durum değişikliği hakkında ayrıntılı bilgi gönderir.

Pull modelinde gözleyiciden gözlenene doğru bir bağımlılık gerekir; bu, bağlaşım açısından bir dezavantaj olarak yorumlanır. Ayrıca gözlenen yalnız "ben değiştim" der; gözleyici durumun neresinin değişip neresinin değişmediğine kendisi bakmak zorundadır. Push modelinde ise gözlenen nesne nelerin değiştiğini bilip yalnız o verileri aktarır; bu sefer yük gözlenene biner. Pull modeli gözlenenin gözleyicilerle bağlantısını azaltır, ama gözleyicilerin gözlenenle bağlantısını artırır.

> **Hocanın vurgusu:** Bağlaşım illa ki olacaktır, ya bir yönde ya öbür yönde; ama iki yönde birden olmaz. Hangisi işimize daha çok yarıyorsa o model seçilir.

Push modelinde aradaki mesaj yükü artar. Hoca bunu bir IoT uygulamasıyla örnekledi: hafif (lightweight) bir sensör ve onun durum değişikliklerini gözleyen bir gözleyici olsun; aralarındaki bağlantı büyük olasılıkla GSM üzerindendir ve yüklenen ya da indirilen her bayt iletişim masrafı olarak yazılır. Bu yüzden mesajların küçük tutulması tercih edilir. Ayrıca gözlenen, içine kod yazılamayan basit bir gömülü sensörse, ayrıntılı bilgi gönderecek işlemci ve belleği yoksa, push modeli kullanılamaz. Gözlenenler daha hafifse gözleyiciye daha çok iş düşen model seçilir. Ama yine de iletişim masrafı söz konusuysa, gözleyicinin işini biraz daha az maliyetle artırıp artıramayacağımıza bakılır; bu, bizi daha büyük bir iletişim faturasından kurtarabilir. Sonuçta bir kâr-zarar analizi yapılıp hangi modelin kullanılacağına karar verilir.

> **Ders sonrası not:** Hoca 02:13:47 civarında hafif gözlenenler için seçilecek modeli bir an "push model" diye adlandırdı. Anlatılan durum (gözlenen yalnız haber verir, işi gözleyici yapar) slayttaki tanıma göre pull modelidir.

8.4 Diğer gerçekleme ayrıntıları

Aynı türden gözlenen nesnelerin tümünün gözleyicilerinin ortak olması istenirse, gözlenen gözleyicilerini statik bir veri yapısında tutabilir (ekrandan, 02:10:00). Yani gerekirse `observers` listesi statik olabilir ve her `ConcreteSubject` bu üyeyi paylaşır. Belli bir türün gözleyicileri hep aynı nesneler olacaksa bu yapılabilir.

Bir gözleyicinin birden fazla nesneyi gözlemesi istenebilir. Ayrıca gözlenen nesnenin durumundaki farklı türden değişiklikleri farklı türden gözleyicilerin izlemesi istenebilir (ekrandan, 02:20:00). Hoca bunu bir pencere örneğiyle anlattı: pencere (`ConcreteSubject`) değişikliklerini izleyen çeşitli gözleyiciler vardır. Bir tür gözleyici yalnız pencereye odak gelip gelmediği ya da odağın kaybedilip kaybedilmediğiyle ilgilenir. Bir başkası pencerenin küçültülmesi, büyütülmesi ya da x veya y ekseninde büyütülmesiyle ilgilenir. Bunların dışındaki olaylarla ilgilenen başka bir gözleyici vardır. Böylece bir pencerenin birden fazla gözleyicisi olur ve bunların bazıları farklı değişikliklerle ilgilenir; belki de kaynak kodu çok karışık tutmamak için böyle ayrılmıştır.

8.5 Haber verme mesajının tetiklenmesi

Önemli bir ayrıntı da haber verme mesajının nasıl ve kim tarafından tetikleneceğidir (ekrandan, 02:20:00):

- Gözleyiciler tetiklerse, gözlenenlerin istekçilerini kodlayan kişi tetikleme komutunu vermeyi unutabilir.
- Gözlenen nesne tetiklerse, gözlenenlerin istekçilerinin peş peşe yapacağı durum değişiklikleri tek bir haber verme mesajıyla gönderilebilecekken birçok izleyiciye gidecek bir sürü mesaj gönderilmesine neden olabilir.

İlk akla gelen çözüm gözlenen nesnenin tetiklemesidir, çünkü durumunun değiştiğinin farkında olan odur. Ama hoca şu durumu örnek verdi: sıcaklık izleyen bir sensör fazla hassas ayarlanmışsa çok küçük sıcaklık değişikliklerinde sürekli güncelleme çalıştırabilir. Her güncelleme, push ya da pull modeli fark etmeksizin veri iletişimi ve masraf demektir. Sıcaklık biraz artıp kısa süre sonra eski değerine dönerse herkese yine güncelleme gider; oysa bu küçük ve hızlı değişiklikler belki önemsizdir.

Gözlenen nesne bunu bir zaman aşımıyla (timeout) yaparsa aralığın doğru belirlenmesi gerekir: çok dar seçilirse yine gereksiz mesaj üretilir, çok geniş seçilirse gözleyiciler önemli bir değişikliği kaçırabilir. Gözleyicinin ara sıra subject'i sorgulaması da düşünülebilir; ama o zaman gözleyicide subject'e giden başvuru zorunlu olur, bu da ek bağlaşım masrafıdır. Eldeki senaryoda en iyi çözüme hangi seçenek götürüyorsa o seçilir.

8.6 Ne zaman kullanılır

Kalıbın kullanılabileceği anlar (ekrandan, 02:20:00):

- Herhangi bir soyutlamanın birbirine bağımlı birden fazla parçası olduğunda (ve bu bağımlılık güncelleme gerektiriyorsa).
- Bir nesnedeki değişikliklerin derleme anında bilinmeyen sayıda nesneyi de etkilemesi gereken anlarda.
- Bağlaşımı arttırmadan bir nesnenin diğer nesneleri herhangi bir durumdan haberdar etmesi istenen anlarda.

9. Observer Örneği: Müşteri İlişkileri Programı

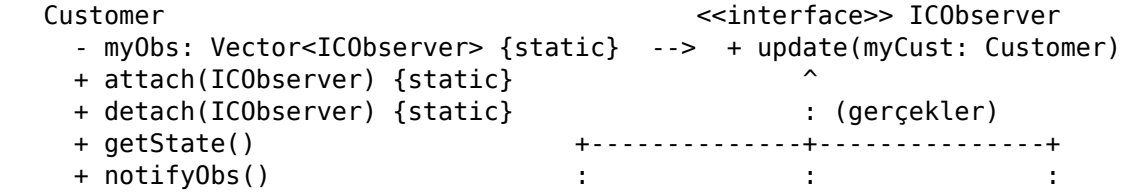
9.1 Problem ve ilk çözüm

Hoca örneğin zor olmadığını, hızlıca geçileceğini söyledi. Bir müşteri ilişkileri programı hazırlıyoruz. Sisteme yeni bir müşteri eklenince müşteriye bir hoş geldiniz mektubu gönderilmeli ve ikamet adresinin doğru, tutarlı, var olan bir adres olup olmadığı denetlenmelidir.

İlk başta bu işler sabit biçimde kodlanabilir: mektup yazmaktan sorumlu bir `WelcomeLetter` sınıfı ve adres doğrulamayla ilgili bir `AddressVerification` sınıfı oluşturulur, `addCustomer` metodu çağrılınca bu sınıfların ilgili metotları çağrılır. Ama gereksinimler değişirse? Pazarlamacılar şöyle fikirlerle gelir: müşteri fiziksel şubelerimizden birine 10 kilometre yakınsa ona göndereceğimiz mektuba bir de indirim kuponu ekleyelim. Bunun gibi bir sürü istek çıkabileceği için daha esnek bir çözüm düşünülür: sisteme her yeni müşteri eklendiğinde bir ya da birkaç farklı işlem yapabilecek bir altyapı. Müşteri eklendiğinde sistem, yapması gereken işler olabileceğinden haberdar edilmeli ve ilgili işleri yapmalıdır. Bu düşünce biçimi Observer kalıbına götürür.

9.2 Observer kalıbı ile çözüm

Çözüm 2: Observer kalıbının kullanılması (ekrandan, 02:24:20):



```
// notifyObs metodu       AddrVerification BrickAndMortar WelcomeLetter
// gözleyicilerin update
// metodunu çağırarak
// onları haberdar eder.
```

`Customer` gözlemlenmesi gereken nesnedir. Onu gözleyecek olanlar `ICObserver` arayüzünde toplanır; adındaki "C" Customer'dan, "Observer" kalıptan gelir. Bu arayüzün bir `update` metodu vardır. `AddrVerification`, `WelcomeLetter` ve fiziksel şubeyi temsil eden `BrickAndMortar` (literatürde fiziksel mağazaya "brick and mortar" denir; şubede indirim kuponu basılacağı zaman bu sınıfın ilgili metodu çalışır) bu arayüzü gerçekler.

Yeni bir müşteri oluşturulunca her zaman eklenecek gözleyiciler kurucuda eklenebilir. Daha sonra yeni kampanyalar başlarsa ya da süresi geçen kampanyalar olursa onların gözleyicilerini eklemek ya da çıkarmak için `attach` ve `detach` metotları vardır. `notifyObs` bütün gözleyicilerin `update` metodunu çağırarak onları haberdar eder. `update` müşterinin kendisini parametre olarak aldığı için bütün veri gönderilmiş olur; hoca bu nedenle çözümün push modeline göre yapıldığını söyledi.

Dikkat edilirse burada soyut bir `Subject` üst sınıfı yoktur; kalıp aslında bunu gerektirir. Slayta göre (ekrandan, 02:25:20) yalnız müşteri nesneleri izlenecekse soyut bir `Subject` üst sınıfı hazırlamaya gerek kalmadan Observer kalıbı kullanılabilir. Ama başka şeyler de izlenecekse, örneğin adres doğrulamanın yalnız müşterileri değil yeni açılan bir şubenin adresini de doğrulaması gerekecekse, soyut bir `Subject` üst sınıfı olmalı ve yeni şube sınıfı da onun somut alt sınıfı olmalıdır. Bu örnek için gerekmediği için o kısım kırpılmıştır.

Hoca sohbette soru gelmediğini görünce dersi bitirdi.

Örnek Sorular

1. (ekrandan, 00:20:00) Bir akıllı ev yazılımında basit bir açma/kapama kumandası ekranı vardır. Kumanda ekranındaki aç/kapa düğmelerine odaların ışıklarını, televizyonları ve benzeri aygıtları açma/kapama işlevleri kazandırılacaktır. Bu problem için nasıl bir tasarım yapılır?

> **Çözüm (derste verildi):** Farklı aygıtların açılıp kapanması farklı biçimlerde olsa da hepsinde açma ve kapama vardır; bu yüzden `ac(): boolean` ve `kapa(): boolean` metotları olan bir `AcKapa` arayüzü tanımlanır. `AydinlatmaDugmesi`, `ElektronikAlet` ve `Iklimlendirme` bu arayüzü gerçekler. `Kumanda` düğmeleri `ArrayList<AcKapa>` içinde tutar; `dugmeEkle`, `ac(dugmeNo)`, `kapa(dugmeNo)` ve `hepsiniKapa()` metotları vardır. `ac(dugmeNo)` düğmeyi bulamazsa `false` döndürür, bulursa düğmenin `ac()` sonucunu iletir. `AnaProgram`'daki `evBoskenSokakKapisiAcildi()` antre ışıklarını ve kombiyi açar, `evdeKimseKalmadi()` bütün düğmeleri kapatır. Böylece kumanda, açma/kapama komutlarını tek bir arayüz üzerinden yürütür.

2. (ekrandan, 00:38:15) Iterator kalıbı koda dökülürken hangi konularda karar verilmelidir?

> **Çözüm (derste verildi):** (1) Parçalar arasında dolaşırken değiştirme işlemine izin verilecek mi? Iterator nesnelerin kendisini gösterirse değişiklik mümkündür; nesnelerin klonları verilirse değişikliğe izin verilmemiş olur. (2) Dolaşım algoritmasını kim gerçekleyecek ve dolaşılacak nesneler birer birer mi, hepsi birden mi verilecek? Ya istekçi gezgine komut verme sırasını belirleyip parçaları tek tek okur, ya da istekçi bir dolaşım istemi verir ve gezgin parçaları bir dizide döndürür. Dolaşım sırası (örneğin ikili ağaçta LDR) bir enum ile istenebilir ve bu seçim veri yapısında ya da iterator'da kodlanabilir. Hangisinin seçileceği problemin doğasına bağlıdır.

3. (ekrandan, 00:44:40 ve 00:56:15) Biri kahvaltılıkta (menüsü `ArrayList` ile), biri yemekte (menüsü dizi ile) uzman iki firma satın alınmıştır. Menü öğeleri ortak, menü gerçeklemeleri farklıdır. Birleştirilmiş menü ve vejetaryen menü müşterilere ortak bir kod üzerinden nasıl sunulur?

> **Çözüm (derste verildi):** `createIterator(): Iterator<MenuItem>` metodu olan bir `Menu` arayüzü tanımlanır; `PancakeHouseMenu` ve `DinerMenu` bunu gerçekler ve sırasıyla `PancakeHouseMenuIterator` ile `DinerMenuIterator` döndürür. Iterator arayüzünde `hasNext(): boolean` ve `next(): MenuItem` vardır. `Waitress` menülerden iterator'ları alır ve tek bir `printMenu(Iterator)` metoduyla "eleman olduğu sürece

sonrakini getir ve yaz" diye dolaşır; vejetaryen menüde aynı dolaşma yalnız vejetaryen öğeleri yazar. Böylece dizi ile `ArrayList` farkı gizlenir ve yeni firmalar için `Waitress`'ın dolaşma kodu değişmez. Kurucunun her yeni firmada değişmemesi için kurucu bir menü listesi ya da değişken sayılı `Menu` parametresi alabilir.

4. (ekrandan, 02:00:00) İç sınıf ile kurulan Memento çözümünde aşağıdaki kod çalıştırılırsa ne olur? Bu engellenebilir mi?

```
Originator fakeSubject = new Originator( );
fakeSubject.modify("sahte durum");
Memento memFake = fakeSubject.createMemento();
subject.setMemento( memFake );
```

Çözüm (derste verildi): Memento, aynı sınıftan bir Originator tarafından oluşturulduğu için geçerli bir mementodur ve `subject` bunu kabul eder; asıl nesnenin durumu "sahte durum" olur. İç sınıf çözümü var olan bir mementonun içeriğinin değiştirilmesini engeller, ama bunu engellemez. Hocanın fikirleri: mementoları Originator'ın kendisinde saklayıp Caretaker'a `activateMemento(int siraNo)` sunmak (ama bu, idareyi Caretaker'a vermek isteyen kalıbın özüne ters düşer) ya da mementoya onu oluşturan nesnenin `hashCode` değerini gömüp Originator'ın yalnız kendi ürettiği mementoyu kabul etmesi (ama bu yalnız program çalıştığı sürece geçerlidir, diske saklanan mementolar programın yeniden açılmasından sonra geçersiz olur). Hoca tam bir çözüm bulamadığını söyledi.

5. (ekrandan, 02:02:20) Belge işleme programındaki `undo()` ve `redo()` metotlarında hangi denetim eksiktir? Nasıl düzeltilir?

Çözüm (derste kısmen verildi): Hoca `undo` içinde `position`'ın sıfırın altına düşmemesi gerektiğini söyledi: `if (position > 0) position--;`.

Çözüm (derste verilmedi): Aynı biçimde `redo` içinde `position`'ın listenin son indisini geçmemesi gerekir: `if (position < states.size() - 1) position++;`. Bu iki denetim python ile küçük bir benzetimle sınandı: iki işlemden sonra üç kez geri alma ilk durumda durur, üç kez yineleme son durumda durur ve indis taşması olmaz.

6. (derste sözlü, 02:12:24) GSM üzerinden bağlı, içine çok kod yazılamayan hafif bir sensörün durum değişikliklerini bir gözleyici izleyecektir. Push mu pull modeli mi seçilmelidir?

Çözüm (derste verildi): İletilen her bayt masraf olduğu için mesajların küçük tutulması tercih edilir. Ayrıca hafif bir sensörün ayrıntılı bilgi gönderecek işlemcisi ve belleği olmayabilir. Bu yüzden işi gözleyiciye bırakan model, yani gözlenenin yalnız haber verdiği ve gözleyicinin durumu kendisinin aldığı pull modeli seçilir. Yine de iletişim masrafı ile gözleyicinin iş yükü arasında bir kâr-zarar analizi yapılarak karar verilmelidir.

7. (ekrandan, 02:24:20) Bir müşteri ilişkileri programında yeni müşteri eklenince hoş geldiniz mektubu gönderilmeli ve adres doğrulanmalıdır; ileride en yakın şubeye 10 km yakın müşterilere indirim kuponu gibi yeni işler eklenecektir. Esnek bir tasarım nasıl kurulur ve bu tasarım push mu pull modeli midir?

Çözüm (derste verildi): Observer kalıbı kullanılır. `Customer`, statik bir `Vector<ICObserver>` içinde gözleyicilerini tutar; statik `attach/detach`, `getState()` ve `notifyObs()` metotları vardır. `ICObserver` arayüzünün `update(myCust: Customer)` metodunu `AddrVerification`, `BrickAndMortar` ve `WelcomeLetter` gerçekler. Müşteri eklenince `notifyObs` her gözleyicinin `update`'ini çağırır. Yeni kampanyalar `attach` ile eklenir, süresi geçenler `detach` ile çıkarılır. `update` müşterinin kendisini aldığı için hoca çözümü push modeli olarak niteledi. Yalnız müşteriler izlendiği için soyut `Subject` üst sınıfına gerek yoktur.

Dersin Toparlaması

Bu derste davranışsal kalıplardan dördü işlendi. Command, istekleri (metot çağrılarını) ortak bir `Execute` arayüzü üzerinden nesnelere çevirir; böylece komutlar kuyruğa alınabilir ve güncesi tutulabilir; invoker ile receiver ayrıdır ve receiver isteğe bağlıdır. Akıllı ev kumandası

örneğinde farklı aygıtlar ortak `AcKapa` arayüzüyle yönetildi ve hoca aynı sınıf yapısının birçok kalıpta görüldüğünü, kalıpları amaçlarının ayırdığını vurguladı. Iterator, bir bütünün parçalarını iç yapısını açmadan dolaşmayı sağlar. Java'nın basit `Iterator`'ı çoğu zaman yeter (KISS), ama dizi ve `ArrayList` kullanan iki menünün tek bir `Waitress` koduyla dolaşıldığı örnek, kalıbın bakım kolaylığını gösterdi.

Memento, bir nesnenin durumunu sarmalamayı bozmadan saklayıp geri yüklemeye yarar. C++'ta friend, Java'da paket erişimi (mühürlü jar gerekir) ya da durumun klonlandığı iç sınıf çözümleriyle Memento'nun durum metotlarına yalnız Originator'ın erişmesi sağlanır; ama aynı sınıftan başka bir nesnenin ürettiği mementonun verilmesi bu çözümlerle tam engellenemedi. Observer, bir nesnenin durum değişikliğinden çalışma anında eklenen gözleyicileri haberdar eder; push ve pull modelleri, statik gözleyici listesi ve haber verme mesajının kimin tetikleyeceği gibi ayrıntılar kâr-zarar analiziyle seçilir. Müşteri ilişkileri örneğinde yeni müşteri eklenince yapılacak işler gözleyiciler olarak eklenip çıkarılabilir hale getirildi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.